

Loop vs Graf – cue cards

12 min · svenska · utvecklare

Kort 0 · Innan du börjar

- Terminal A och B öppna i repot, `claude` startat i båda. `npm run reset` körd.
- Artifact öppen i Chrome, `F` för fullskärm, presenter-panelen AV.
- Nät? Om nej: öppna `demo/runs/` – kör bara artifacten.
- Klocka på. Mål 12 min.

Tangenter: `←/→` scen · Space spela · R om · N stödord · F fullskärm · E lägg till kant (scen 4)

Kort 1 • Hook

0:00–1:00 • Scen 1, stilla

SÄG

Samma uppgift: triagera fem supportärenden. Två sätt att bygga agenten. Loop: en prompt och ett stoppvillkor. Graf: jag ritar noderna själv. Frågan är inte vilket som är bäst – utan vad du vet i förväg.

GÖR

Ingenting ännu.

Kort 2 • Loopen

1:00–3:00 • Scen 1, Space

SÄG

Prompt + stoppvillkor. Modellen väljer vägen. Tänk → Agera → Observera → Klar?
Claude Code är redan en loop – Ralph gör loopen yttre och explicit: samma prompt igen tills promisen är sann. Min artefakt är stoppvillkoret, inte vägen.

GÖR

Terminal A:

```
/ralph-loop:ralph-loop Triagera alla tickets enligt TRIAGE.md --completion-promise "ALLA TICKETS TRIAGERADE" --max-iterations 8
```

Enter. Låt köra. Tillbaka till artifacten.

OM DET GÅR FEL

Visa `demo/runs/loop-run.md` .

Kort 3 • Grafen

3:00–5:00 • Scen 2, Space

SÄG

Jag ritar noderna: Inbox → Klassificera → Kund || Policy → Svara eller Eskalera → Verifiera. Varje nod är ett bundet agentanrop med schema. Kanterna är kod. Parallellism gratis. Faserna syns live.

GÖR

Terminal B: `/trriage-graph` Enter. Peka på faserna. Tillbaka.

OM DET GÅR FEL

Visa `demo/runs/graph-run.md` .

Kort 4 · Sida vid sida

5:00–6:30 · Scen 3, Space

SÄG

Fyra vanliga ärenden. Båda gröna. Loopen tog ett antal steg jag inte visste i förväg. Grafen tog exakt så många som jag ritade. Kostnad: loop okänd, graf bunden.

Kort 5 • Edge case

6:30–8:00 • Scen 4, Space, sen E

SÄG

Ticket fem. Grekiska. GDPR. Ingen kategori passar. Loopen: check säger nej, ett varv till, landar på other + eskalera. Grafen: Klassificera säger "other" – och det finns ingen kant. Stopp. (E) Kanten du lägger till efter att den bitit dig. Grafen kräver att du såg det komma. Loopen kräver att du litar på modellen.

Kort 6 · Tillbaka till terminalerna

8:00–10:00

GÖR

Terminal A: scrolla, visa varven. `npm run check:loop`. Terminal B: fasloggen. `npm run check:graph`. Sen `npm run diff`.

SÄG

Två gröna. Två helt olika spår. Loopens spår läser du i efterhand. Grafens spår ritade du i förväg.

OM DET GÅR FEL

`npm run diff` funkar på förkörda filer.

Kort 7 • Hybrid + när använda vad

10:00–12:00 • Scen 5 Space, sen scen 6 Space

SÄG

Zooma in i en nod – där sitter en loop. Varje agent()-nod i Workflow ÄR en Claude Code-loop. Struktur utanpå, frihet inuti. Tabellen. Takeaway: Graf för det du vet. Loop för det du inte vet. Oftast: graf med loopar i noderna.

FRÅGA TILL RUMMET

Var i era pipelines har ni loopar som borde vara grafer – och grafer som borde vara loopar?